

Playwright Entity Question Validation – Complete Code

test-data/questionEntityMapping.json

```
[
  {
    "question": "Are any of the securities pledged as collateral restricted or control securities?",
    "entityTypes": ["Individual", "Corporation", "Joint", "NoProfit"]
  },
  {
    "question": "Does this limit request involve the transfer of an existing debit balance?",
    "entityTypes": ["Individual", "Corporation", "Joint", "NoProfit"]
  },
  {
    "question": "Is the account holder a Native American Tribe or Tribal Corporation?",
    "entityTypes": ["Individual", "Corporation", "Joint", "NoProfit"]
  },
  {
    "question": "Is the account holder a non-profit entity?",
    "entityTypes": ["NoProfit"]
  }
]
```

enums/EntityType.ts

```
export enum EntityType {
  INDIVIDUAL = 'Individual',
  CORPORATION = 'Corporation',
  JOINT = 'Joint',
  NO_PROFIT = 'NoProfit'
}
```

utils/QuestionEntityValidator.ts

```
import fs from 'fs';
import path from 'path';
import { EntityType } from '../enums/EntityType';

interface QuestionMapping {
  question: string;
  entityTypes: EntityType[];
}

export class QuestionEntityValidator {

  private static mapping: QuestionMapping[] = JSON.parse(
    fs.readFileSync(
      path.join(__dirname, '../test-data/questionEntityMapping.json'),
      'utf-8'
    )
  );

  static getExpectedQuestions(entityType: EntityType): string[] {
    return this.mapping
      .filter(q => q.entityTypes.includes(entityType))
      .map(q => q.question);
  }

  static validateUIQuestions(uiQuestions: string[], entityType: EntityType) {
    const expectedQuestions = this.getExpectedQuestions(entityType);
    const missingOnUI = expectedQuestions.filter(
```

```

        q => !uiQuestions.some(uiQ => uiQ.includes(q))
    );

    const extraOnUI = uiQuestions.filter(
        uiQ => !expectedQuestions.some(eq => uiQ.includes(eq))
    );

    return {
        expectedQuestions,
        missingOnUI,
        extraOnUI
    };
}
}

```

tests/entity-question-validation.spec.ts

```

import { test, expect } from '@playwright/test';
import { EntityType } from '../enums/EntityType';
import { QuestionEntityValidator } from '../utils/QuestionEntityValidator';

test('Validate all questions for selected entity type', async ({ page }) => {
    const selectedEntity = EntityType.NO_PROFIT;

    await page.goto('/your-page-url');

    const uiQuestions = await page
        .locator('table tr td:nth-child(1)')
        .allTextContents();

    const { missingOnUI, extraOnUI } =
        QuestionEntityValidator.validateUIQuestions(
            uiQuestions,
            selectedEntity
        );

    expect(missingOnUI, 'Missing questions on UI').toEqual([]);
    expect(extraOnUI, 'Extra questions on UI').toEqual([]);
});

```